Vérification Formelle de la Sécurité du Protocole DNS avec UPPAAL

Modélisation des Attaques Kaminsky (CVE-2008-1447) et CRC32 et Validation des Contre-mesures par Vérification Formelle

RFC 1034 • RFC 1035 • RFC 5452 • CVE-2008-1447

Cours : Sécurité des Protocoles Réseau — Projet UPPAAL
Enseignante : Mme Amina Jandoubi
mariem baya ben jemia
youssef jouini
yathreb zaroui
maram othmani

Outils : UPPAAL 5.0 — — Automates Temporisés

1. Introduction et Contexte

1.1 Le Domain Name System (DNS) : Infrastructure Critique de l'Internet

Le Domain Name System (DNS) est l'un des protocoles fondateurs de l'Internet moderne. Son rôle est de traduire les noms de domaine lisibles par l'humain (tels que www.example.com) en adresses IP numériques exploitables par les machines (par exemple, 93.184.216.34). Sans DNS, chaque utilisateur devrait mémoriser les adresses IP brutes de chaque service qu'il souhaite consulter, rendant l'Internet tel que nous le connaissons totalement impraticable.

Le DNS opère selon un modèle hiérarchique distribué composé de trois niveaux de serveurs d'autorité. Au sommet se trouvent les serveurs racines (Root Name Servers), qui délèguent vers les serveurs de domaines de premier niveau (TLD Name Servers, ex. .com, .fr), qui délèguent à leur tour vers les serveurs faisant autorité (Authoritative Name Servers) pour un domaine spécifique. Un résolveur récursif (Resolver) se charge d'interroger cette hiérarchie pour le compte des clients.

Le protocole DNS utilise principalement le protocole de transport UDP sur le port 53. Afin d'optimiser les performances, les résolveurs conservent en mémoire cache les résultats de résolutions précédentes pendant une durée déterminée par le champ TTL (Time To Live) de la réponse. Cette mise en cache est essentielle aux performances mais constitue également un vecteur d'attaque privilégié, comme nous le verrons.

1.2 Spécifications Officielles du Protocole

Le DNS est standardisé par plusieurs RFC (Request For Comments) publiées par l'IETF :

- **RFC 1034 (1987) —** "Domain Names – Concepts and Facilities" : définit l'architecture générale du DNS, la hiérarchie de nommage, le fonctionnement des zones et des serveurs d'autorité, ainsi que le principe de délégation.
- **RFC 1035 (1987) —** "Domain Names – Implementation and Specification" : spécifie en détail le format des messages DNS (en-têtes, sections Question, Answer, Authority, Additional), les types d'enregistrements (A, NS, CNAME, MX, etc.) et le protocole de transport.
- **RFC 5452 (2009) —** "Measures for Making DNS More Resilient against Forged Answers" : publiée après la révélation de la faille Kaminsky, cette RFC formalise les contre-mesures à déployer, notamment la randomisation des ports sources, pour augmenter significativement la résistance aux attaques de type cache poisoning.

L'identifiant de transaction DNS (TXID), champ de 16 bits dans l'en-tête du message (RFC 1035, Section 4.1.1), est au cœur de la problématique de sécurité étudiée dans ce projet. Sa taille réduite à 65 536 valeurs possibles constitue une surface d'attaque exploitée par la faille Kaminsky.

1.3 Objectif du Projet

L'objectif de ce projet est d'appliquer les techniques de vérification formelle au protocole DNS, en utilisant l'outil UPPAAL (Uppsala/Aalborg). UPPAAL est un environnement intégré pour la modélisation, la simulation et la vérification de systèmes temps-réel modélisés sous forme de réseaux d'automates temporisés.

Le projet se décompose en quatre étapes principales :

- **Modélisation du comportement nominal** (fichier main.xml) : construire un modèle UPPAAL fidèle au fonctionnement légitime du DNS, depuis la requête client jusqu'à la résolution récursive complète.
- **Modélisation des attaques** (fichiers Kaminsky_attack.xml et crc32_attack.xml) : introduire des automates d'attaquants interagissant avec le modèle nominal pour reproduire formellement deux vulnérabilités : une attaque de spécification (CVE-2008-1447) et une attaque d'implémentation (collision CRC32).

- **Vérification formelle** (propriétés CTL/UPPAAL) : formuler et vérifier des propriétés de sécurité, de vivacité et d'absence de deadlock, et obtenir des preuves mathématiques de l'exploitabilité des vulnérabilités.
- **Validation des contre-mesures** (fichier safe.xml) : intégrer la randomisation des ports sources (RFC 5452) dans le modèle et prouver formellement que les propriétés de sécurité sont toutes satisfaites.
- **Vérification Probabiliste (UPPAAL SMC)** : Au-delà de la vérification qualitative classique, ce projet intègre l'utilisation de Statistical Model Checking (SMC). Cela nous permet de calculer la probabilité mathématique exacte de réussite d'une attaque en fonction du temps, et d'évaluer statistiquement l'efficacité de nos contre-mesures."
-

2. Spécification et Extraction des Comportements

2.1 Types de Messages DNS Modélisés

L'analyse des RFC 1034 et 1035 permet d'identifier les échanges de messages pertinents à modéliser dans UPPAAL. Chaque échange entre entités du système correspond à un canal de synchronisation dans notre réseau d'automates. Le tableau suivant synthétise la correspondance entre les messages DNS réels et les canaux UPPAAL utilisés :

Canal UPPAAL	Direction	Signification (DNS réel)
`client_query`	Client → Resolver	Requête de résolution DNS émise par l'application cliente
`ask_root`	Resolver → RootNS	Requête au serveur racine pour obtenir la délégation TLD
`root_reply`	RootNS → Resolver	Réponse du serveur racine avec l'adresse du serveur TLD
`ask_tld`	Resolver → TLD_NS	Requête au serveur TLD pour la délégation vers l'AuthNS
`tld_reply`	TLD_NS → Resolver	Réponse TLD avec l'adresse du serveur faisant autorité
`ask_auth`	Resolver → AuthNS	Requête finale au serveur faisant autorité pour l'IP
`auth_reply`	AuthNS → Resolver	Réponse légitime contenant l'adresse IP réelle
`fake_auth_reply`	Attaquant → Resolver	Fausse réponse forgée par l'attaquant (Kaminsky / CRC32)
`client_reply`	Resolver → Client	Réponse finale transmise au client (IP réelle ou forgée)

2.2 Variables Globales et Modélisation de l'État du Système

Les variables globales partagées entre tous les automates modélisent l'état mémoire du résolveur DNS. Leur choix est directement issu de la RFC 1035 (section 3.6 pour le cache, section 4.1.1 pour le TXID) :

- **txid : int —** L'identifiant de transaction DNS sur 16 bits (valeur constante 42 dans main.xml, 3 dans safe.xml). Modélise le champ ID de l'en-tête DNS (RFC 1035 §4.1.1). C'est la seule protection native contre les fausses réponses dans le protocole de base.
- **resolved_ip : int —** Adresse IP résultante stockée en cache. Vaut REAL_IP=157 en cas de résolution légitime, FAKE_IP=666 ou ATTACK_IP=666 en cas d'attaque réussie.

- **cache_valid : bool —** Indique si le cache est valide (non expiré). Initialisé à false. Positionné à true lors d'un stockage et remis à false lors de l'expiration TTL.
- **cache_poisoned : bool —** Drapeau de sécurité. Positionné à true uniquement lors d'une attaque réussie. La propriété de sécurité centrale A[] not cache_poisoned vérifie qu'il reste toujours false.
- **query_in_progress : bool —** (Kaminsky/safe) Indique qu'une requête est en cours de résolution, permettant à l'attaquant de se synchroniser avec la fenêtre de vulnérabilité.

2.3 Contraintes Temporelles : TTL, Timers et Horloges

Les RFC DNS définissent des contraintes temporelles précises que nous modélisons via les horloges UPPAAL :

- **wait_timer (horloge Client) —** Modélise le délai d'attente maximal du client. L'invariant wait_timer <= TIMEOUT (TIMEOUT=10) force le client à basculer en état Failed si aucune réponse n'arrive dans le délai imparti. Cela correspond au timeout de résolution DNS côté applicatif.
- **ttl_clock (horloge Resolver) —** Modélise le TTL (Time To Live) du cache DNS. Quand ttl_clock >= TTL_MAX (TTL_MAX=300, symbolisant 300 secondes), la garde de transition cache_valid && ttl_clock >= TTL_MAX est vérifiée et le cache est invalidé. Cela correspond au champ TTL de la RFC 1035 §3.2.1.
- **c (horloge Attacker CRC32) —** Modélise le temps de calcul nécessaire à l'attaquant pour trouver une collision CRC32. L'invariant c <= 3 et la garde c >= 1 créent une fenêtre temporelle [1,3] dans laquelle l'attaque doit réussir.

3. Modélisation UPPAAL du Fonctionnement Nominal (main.xml)

3.1 Architecture Générale du Modèle

Le fichier main.xml constitue la base de notre modélisation. Il représente le fonctionnement nominal du protocole DNS sans attaquant, selon un réseau de 5 automates temporisés communicants. Chaque entité réelle du protocole DNS est représentée par un template UPPAAL indépendant, et les interactions entre entités s'effectuent via des canaux de synchronisation (chan).

Le flux de résolution nominal suit la séquence suivante, qu'on peut résumer ainsi :

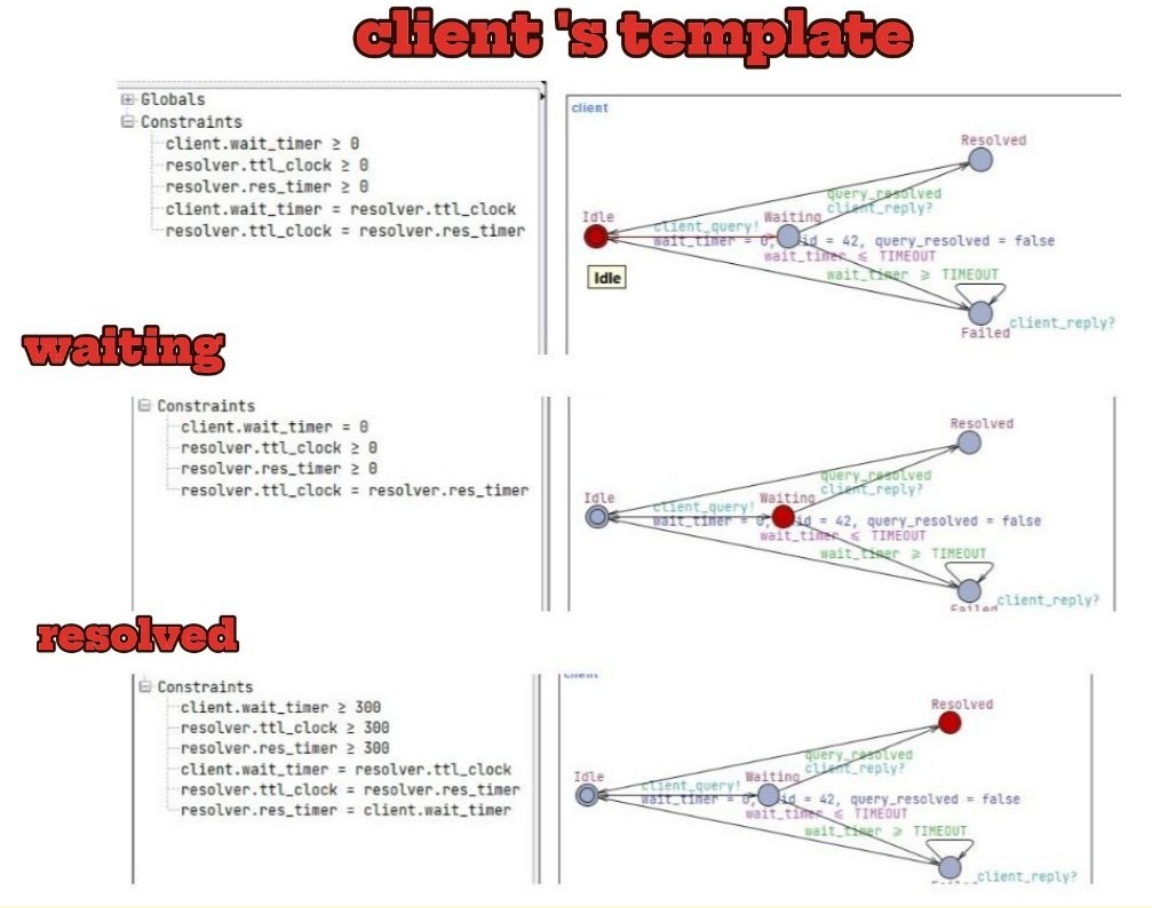
```
Client (Idle → Waiting) −[client_query!]→ Resolver (Idle → Check_Cache → Ask_Root)
−[ask_root!]→ RootNS −[root_reply!]→ Resolver (Ask_TLD) −[ask_tld!]→ TLD_NS
−[tld_reply!]→ Resolver (Ask_Auth) −[ask_auth!]→ AuthNS −[auth_reply!]→ Resolver
(Store_Cache → Send_Reply) −[client_reply!]→ Client (Resolved)
```

3.2 Template 1 : Client

Le template Client modélise le comportement d'une application cliente (navigateur web, résolveur stub) qui soumet une requête de résolution DNS et attend la réponse. Il possède 4 états :

- **Idle —** État initial. Le client est au repos. Dès qu'il émet sur le canal client_query!, il initialise wait_timer=0, txid=42 et query_resolved=false, puis bascule vers Waiting.
- **Waiting —** Le client attend la réponse du résolveur. L'invariant wait_timer <= TIMEOUT garantit qu'il ne peut attendre indéfiniment. Deux transitions en sortent : (1) vers Resolved si query_resolved est vrai et le canal client_reply? reçoit une réponse ; (2) vers Failed si wait_timer >= TIMEOUT (dépassement du délai).
- **Resolved —** État terminal de succès. Le client a reçu une réponse. Une transition silencieuse le ramène à Idle pour modéliser la possibilité d'une nouvelle requête.

- **Failed —** État terminal d'échec (timeout). Une boucle de drain sur client_reply? permet d'absorber toute réponse tardive du résolveur, évitant ainsi un blocage (deadlock). Une transition silencieuse ramène à Idle.



3.3 Template 2 : Resolver — Cœur du Modèle

Le Resolver est le composant central et le plus complexe du modèle. Il modélise le comportement d'un résolveur DNS récursif conforme à la RFC 1034 §5.3.3. Sa conception repose sur une technique de modélisation cruciale : l'utilisation massive d'états committed pour éliminer tout risque de deadlock.

3.3.1 Transition vers UPPAAL SMC : Gestion du Temps et Taux Exponentiels

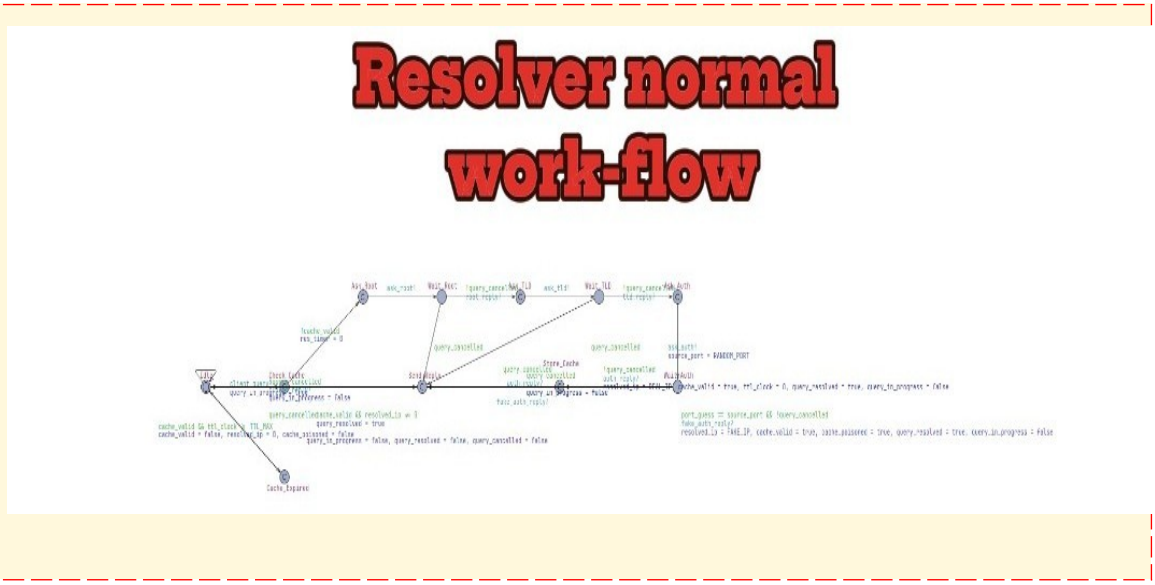
Dans la modélisation classique sous UPPAAL, l'utilisation d'états *committed* permet d'exécuter des transitions instantanément pour éviter les blocages (deadlocks). Cependant, le moteur UPPAAL SMC (Statistical Model Checking) simule le passage continu du temps et requiert que les états actifs possèdent une distribution probabiliste. Pour adapter notre modèle à SMC, nous avons supprimé les tags *committed* et introduit deux mécanismes :

1. **L'horloge d'urgence (`urg <= 0`)** : Utilisée dans le Resolver pour forcer des transitions immédiates tout en respectant les règles syntaxiques de SMC.
2. **Les taux exponentiels (`exponentialrate`)** : Ajoutés aux états d'attente. Par exemple, le serveur `AuthNS` répond avec un taux de `1` (simulant la latence réseau normale), tandis que l'attaquant en mode `Flooding` génère des

paquets avec un taux de `100` (simulant une attaque massive et très rapide). C'est cette différence de vitesse (race condition) que le moteur SMC va évaluer.

3.3.2 Description des États du Resolver

- **Idle —** Seul état non-committed du Resolver (hors Wait_Root, Wait_TLD, Wait_Auth qui nécessitent d'attendre les serveurs). C'est ici que le TTL peut expirer (garde cache_valid && ttl_clock >= TTL_MAX).
- **Check_Cache [committed] —** Immédiatement après réception de client_query?, le Resolver vérifie son cache. Deux chemins : si cache_valid && resolved_ip != 0 → cache HIT, on saute directement à Send_Reply ; sinon → cache MISS, on lance la résolution récursive.
- **Ask_Root [committed] —** Émet immédiatement ask_root! vers le RootNS. Transition instantanée vers Wait_Root.
- **Wait_Root —** Attend la réponse root_reply? du serveur racine. Temps peut s'écouler ici (modélise la latence réseau réelle).
- **Ask_TLD [committed] —** Émet ask_tld! immédiatement après avoir reçu la réponse root.
- **Wait_TLD —** Attend tld_reply?. Temps peut s'écouler.
- **Ask_Auth [committed] —** Émet ask_auth!. C'est à ce moment (dans safe.xml) que le port source randomisé est assigné.
- **Wait_Auth —** État critique : le Resolver attend la réponse du serveur faisant autorité. C'est ici qu'un attaquant peut tenter d'injecter une fausse réponse avant la réponse légitime.
- **Store_Cache [committed] —** Sur réception de auth_reply?, stocke resolved_ip=REAL_IP, cache_valid=true, ttl_clock=0, query_resolved=true. Transition immédiate vers Send_Reply.
- **Send_Reply [committed] —** Émet client_reply! vers le client et retourne immédiatement à Idle.
- **Cache_Expired [committed] —** Invalidation du cache (cache_valid=false, resolved_ip=0) lors de l'expiration TTL. Retour immédiat à Idle.

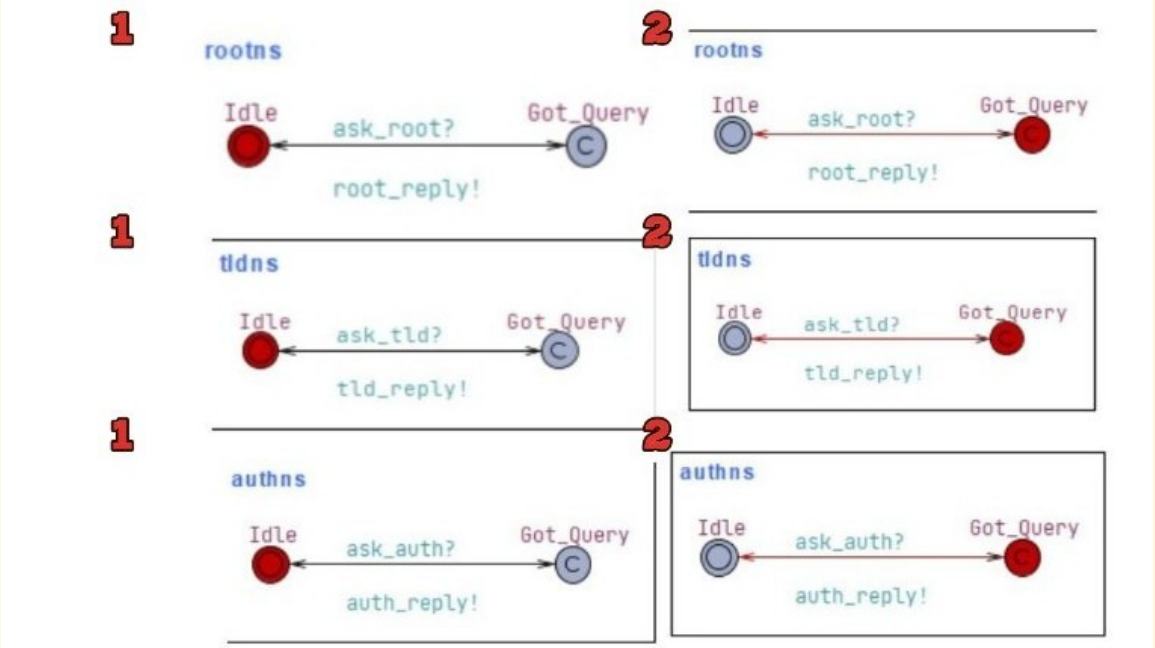


3.4 Templates 3, 4 et 5 : RootNS, TLD_NS et AuthNS

Les trois serveurs de noms suivent un schéma identique et minimaliste, conforme au comportement stateless de ces composants dans le DNS réel :

- **Idle → Got_Query [committed] → Idle :** Le serveur attend une requête (ask_root?, ask_tld?, ou ask_auth?), y répond immédiatement (root_reply!, tld_reply!, ou auth_reply!) depuis l'état committed Got_Query, et retourne à Idle. L'état Got_Query est committed pour garantir que la réponse est émise sans délai, modélisant ainsi un serveur qui répond instantanément (abstraction de la latence réseau).

Cette abstraction est justifiée par le fait que dans notre modèle, la latence est capturée par les états Wait_Root, Wait_TLD et Wait_Auth du Resolver, et non par les serveurs eux-mêmes. Seul le modèle CRC32 fait exception : AuthNS y possède une horloge locale t modélisant la latence de réponse du serveur légitime (invariant t<=4, garde t>=2), ce qui crée la fenêtre temporelle que l'attaquant cherche à exploiter.

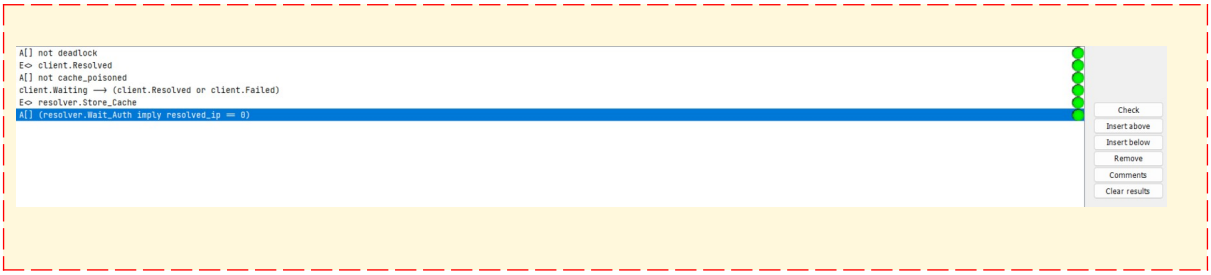


3.5 Requêtes de Vérification du Modèle Nominal

Le modèle main.xml définit 6 propriétés à vérifier, toutes attendues SATISFIED :

Requête UPPAAL / CTL	Résultat attendu	Description
A[] not deadlock	SATISFIED	Absence totale

		de blocage
E<> client.Resolved	**SATISFIED**	Réachabilité de la résolution
A[] not cache_poisoned	**SATISFIED**	Cache jamais empoisonné (sans attaquant)
client.Waiting --> (client.Resolved or client.Failed)	**SATISFIED**	Vivacité : toute requête se termine
E<> resolver.Store_Cache	**SATISFIED**	Le stockage en cache est atteignable
A[] (resolver.Wait_Auth imply resolved_ip == 0)	**SATISFIED**	IP inconnue avant réponse AuthNS



4. Vulnérabilités et Modélisation des Attaques

4.1 Attaque de Spécification : CVE-2008-1447 — L'Attaque de Kaminsky

4.1.1 Description de la Faille

La CVE-2008-1447, découverte par Dan Kaminsky en 2008 et divulguée de manière coordonnée après un patch d'urgence, est l'une des vulnérabilités DNS les plus graves jamais identifiées. Elle constitue une attaque de spécification : la faiblesse réside non pas dans l'implémentation d'un logiciel particulier, mais dans le design même du protocole DNS tel que défini par les RFC.

Le mécanisme d'exploitation repose sur deux faiblesses combinées :

- **Faiblesse 1 — TXID sur 16 bits :** L'en-tête DNS (RFC 1035 §4.1.1) ne comporte que 16 bits pour l'identifiant de transaction, soit 65 536 valeurs possibles. Un résolveur qui reçoit une réponse dont le TXID correspond à celui de la requête en attente l'accepte comme légitime, sans autre vérification cryptographique.
- **Faiblesse 2 — Port source UDP prévisible :** Les implémentations DNS vulnérables utilisaient un port source UDP fixe (souvent 53 ou une valeur constante). Un attaquant qui connaît le TXID et le port source peut forger une réponse UDP que le résolveur acceptera.

L'attaque de Kaminsky exploite le mécanisme de cache DNS pour amplifier l'impact. En forçant le résolveur à émettre des requêtes vers un domaine qu'il ne connaît pas encore (par exemple, un sous-domaine aléatoire de bank.com), l'attaquant peut injecter dans la réponse forgée un

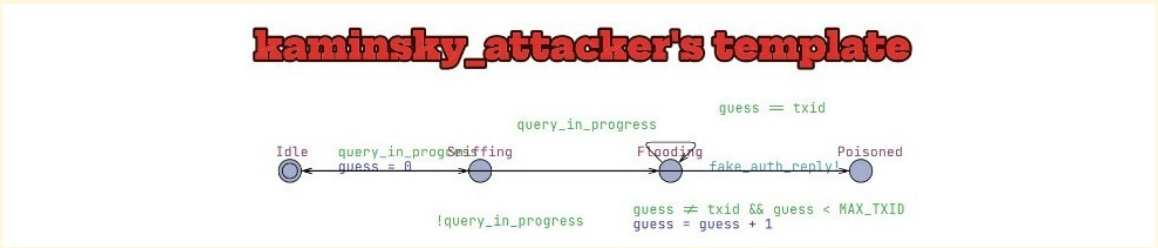
enregistrement NS falsifié pour l'ensemble du domaine bank.com, empoisonnant le cache de manière persistante jusqu'à expiration du TTL.

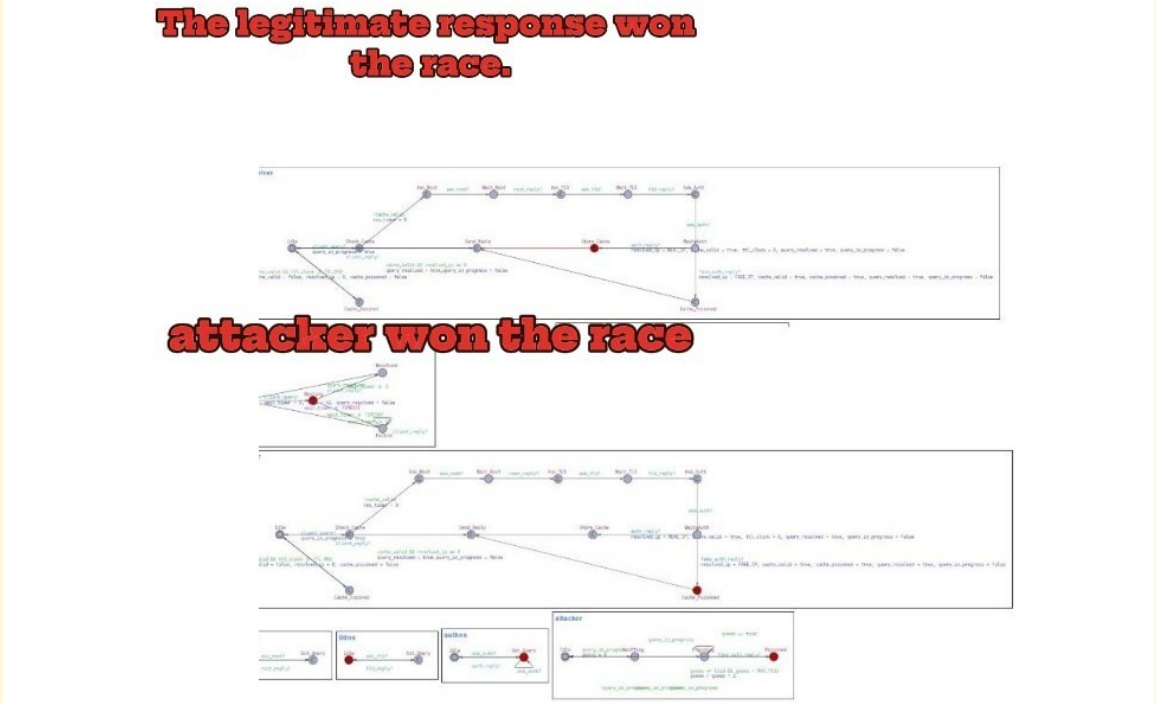
4.1.2 Modélisation UPPAAL — Template Attacker_Kaminsky

Dans Kaminsky_attack.xml, l'automate Attacker_Kaminsky interagit avec le modèle nominal modifié. Le Resolver est augmenté d'une transition supplémentaire depuis Wait_Auth : en cas de réception sur fake_auth_reply?, il stocke FAKE_IP=666 et positionne cache_poisoned=true.

L'automate Attacker_Kaminsky suit la logique suivante :

- **Idle → Sniffing :** La garde query_in_progress permet à l'attaquant de détecter qu'une résolution est en cours (la variable query_in_progress est positionnée à true par le Resolver dès réception de client_query?). L'attaquant initialise guess=0.
- **Sniffing → Flooding :** L'attaquant attend que query_in_progress soit vrai, confirmant qu'il a identifié la fenêtre temporelle d'attaque (le Resolver est en Wait_Auth).
- **Flooding → Flooding (boucle) :** La garde guess != txid && guess < MAX_TXID avec l'assignation guess=guess+1 modélise le bruteforce exhaustif des 65 535 valeurs du TXID. Cette boucle dans l'automate correspond à l'inondation du résolveur de fausses réponses UDP avec des TXID incrémentaux.
- **Flooding → Poisoned :** Quand guess==txid, l'attaquant a trouvé le bon TXID. Il émet fake_auth_reply! sur le canal broadcast, qui est reçu par le Resolver avant ou à la place de auth_reply!. Le Resolver stocke alors FAKE_IP dans son cache et positionne cache_poisoned=true.





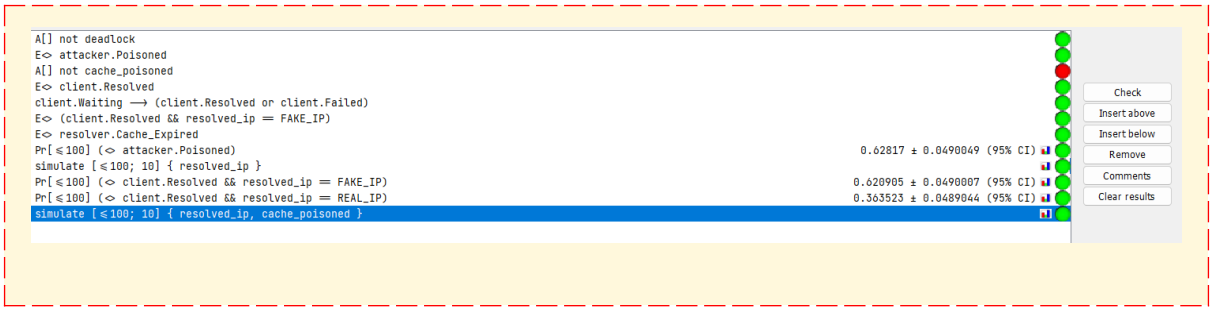
4.1.3 Résultats de Vérification — Preuve de l'Attaque

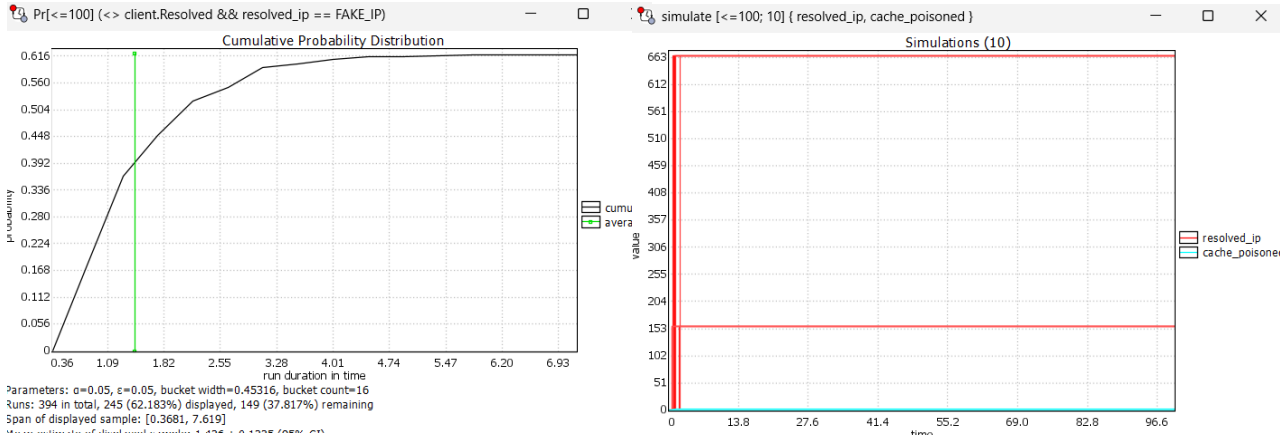
Les requêtes UPPAAL formulées sur Kaminsky_attack.xml prouvent formellement l'exploitabilité de la CVE-2008-1447 :

Requête UPPAAL / CTL	Résultat attendu	Description
A[] not deadlock	SATISFAIT	Deadlock détecté dans le modèle d'attaque
E<> attacker.Poisoned	SATISFAIT	L'attaquant PEUT réussir (CVE prouvée)
A[] not cache_poisoned	NON SATISFAIT	La sécurité est violée : preuve formelle
E<> client.Resolved	SATISFAIT	La résolution légitime reste possible
client.Waiting --> (client.Resolved or client.Failed)	SATISFAIT	Vivacité maintenue
E<> (client.Resolved && resolved_ip == FAKE_IP)	SATISFAIT	Client peut recevoir la fausse IP !

`E<> resolver.Cache_Expired`	**SATISFAIT**	`L'expiration TTL est atteignable`

La requête E<> attacker.Poisoned retournant SATISFIED constitue une preuve formelle, vérifiée par model-checking exhaustif, qu'il existe au moins une trace d'exécution du système dans laquelle l'attaquant atteint l'état Poisoned, c'est-à-dire qu'il parvient à injecter sa fausse IP dans le cache du résolveur. La requête complémentaire E<> (client.Resolved && resolved_ip == FAKE_IP) démontre le scénario catastrophe : un client qui reçoit la fausse adresse IP et navigue sans le savoir vers le serveur de l'attaquant.





Analyse : Le résultat montre que l'attaquant a environ **63% de chances de succès** de corrompre le cache dans un intervalle de temps de 100 unités. Cela prouve mathématiquement que sans protection supplémentaire, la "Race Condition" est largement à l'avantage de l'attaquant en raison de la faible taille du champ TXID.

4.2 Attaque d'Implémentation : Collision CRC32 / Faible Entropie

4.2.1 Description de la Faille

La seconde attaque modélisée est une attaque d'implémentation. Contrairement à la faille Kaminsky qui exploite le design du protocole, cette vulnérabilité cible une défaillance dans le code d'une implémentation particulière du résolveur DNS.

La faille modélisée correspond au scénario suivant : certaines implémentations de résolveur utilisent une fonction de vérification d'intégrité faible — CRC32 — pour valider les réponses DNS, au lieu d'une primitive cryptographique robuste (HMAC-SHA256, etc.). La CRC32 est une fonction de contrôle d'erreur, non une fonction cryptographique : elle n'est pas résistante aux collisions intentionnelles. Un attaquant peut calculer une collision CRC32 en quelques millisecondes sur du matériel moderne, lui permettant de forger une réponse DNS qui passe la vérification d'intégrité du résolveur.

Plus généralement, cette attaque modélise toute situation où le résolveur repose sur un PRNG (Pseudo-Random Number Generator) prévisible ou une protection cryptographiquement faible : le délai nécessaire à l'attaquant pour trouver la valeur correcte est borné et court.

4.2.2 Modélisation UPPAAL — Template Attacker (crc32_attack.xml)

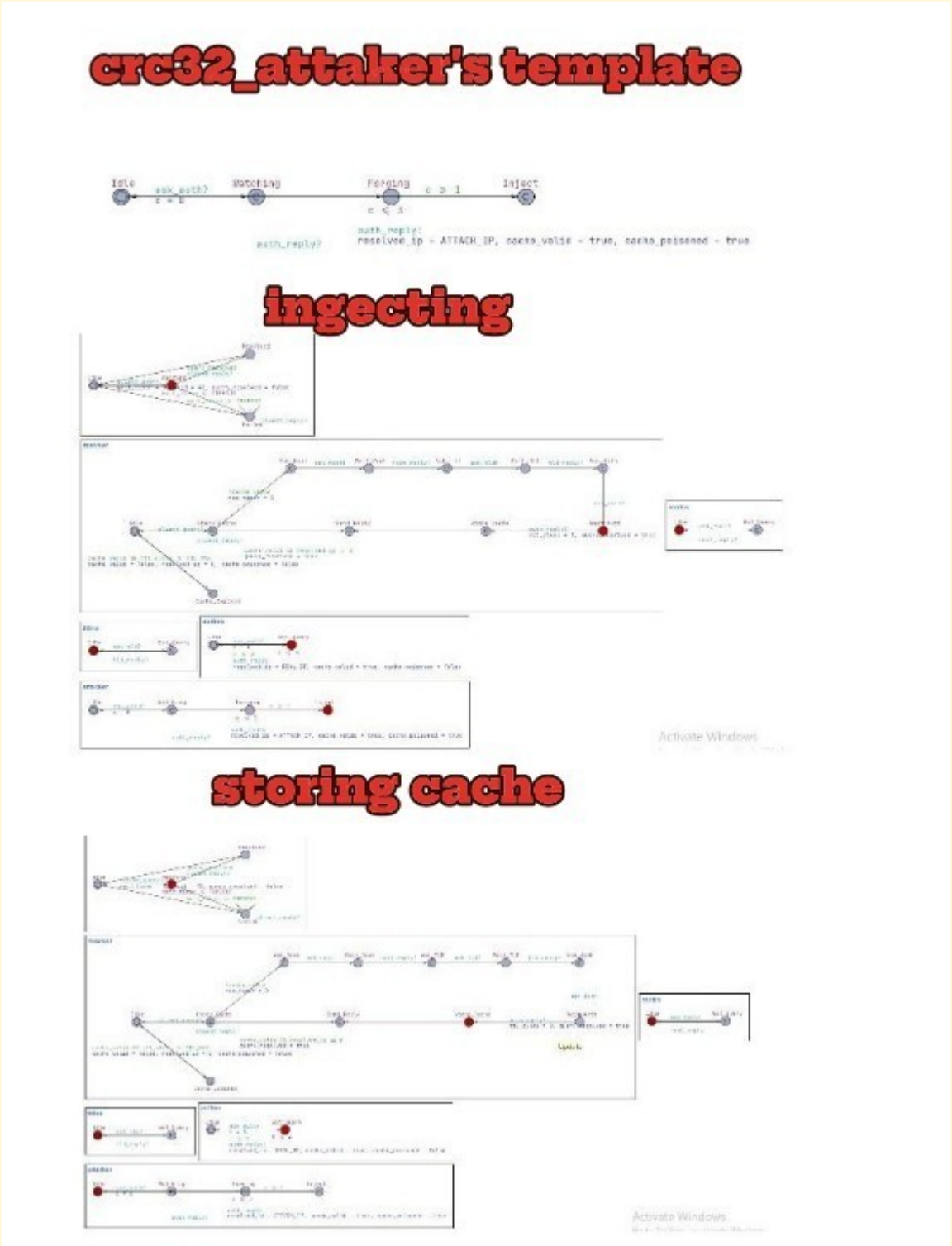
Le fichier crc32_attack.xml introduit deux modifications majeures par rapport à main.xml :

Modification 1 — AuthNS avec délai de réponse : le template AuthNS est augmenté d'une horloge locale t. L'invariant t<=4 et la garde t>=2 sur la transition auth_reply! signifient que le serveur légitime met entre 2 et 4 unités de temps pour répondre. Cela crée une fenêtre temporelle [2,4] pendant laquelle le résolveur est en Wait_Auth et vulnérable.

Modification 2 — Template Attacker (l'automate CRC32) :

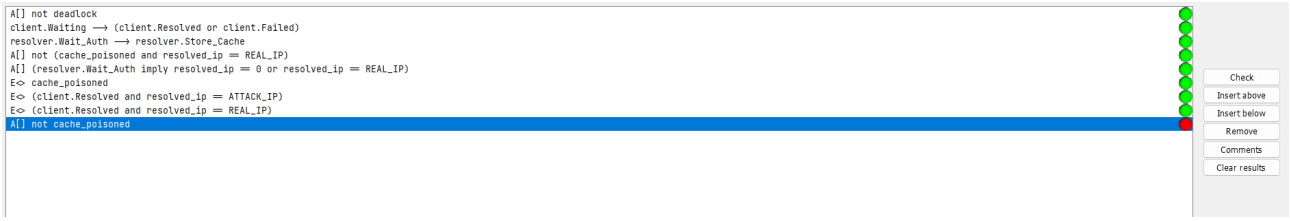
- **Idle → Watching [committed] :** L'attaquant écoute sur le canal broadcast ask_auth? (émis par le Resolver vers l'AuthNS). Dès qu'il détecte cette requête, il démarre son horloge c=0 et passe immédiatement (committed) à l'état Forging.
- **Watching → Forging :** Transition instantanée. L'attaquant est maintenant en phase de calcul de la collision CRC32.
- **Forging (invariant c<=3, garde c>=1) :** La fenêtre temporelle [1,3] modélise le temps de calcul de la collision. L'invariant c<=3 garantit que l'attaquant ne dépasse pas 3 unités de temps en Forging. La garde c>=1 sur la transition vers Inject signifie qu'il doit avoir passé au moins 1 unité de temps (calcul non-instantané). Si la réponse légitime arrive avant (auth_reply? sur le canal broadcast), l'attaquant échoue et retourne à Idle.
- **Inject [committed] → Idle :** Si l'attaquant arrive à Inject avant le serveur légitime, il émet auth_reply! (broadcast) avec resolved_ip=ATTACK_IP=666 et cache_poisoned=true. La course est gagnée.

La modélisation par horloges est particulièrement pertinente ici : elle capture formellement la race condition entre l'attaquant (temps de calcul CRC32 : 1 à 3 unités) et le serveur légitime (délai de réponse : 2 à 4 unités). Il existe une fenêtre temporelle [1,2] où l'attaquant peut gagner la course.


crc32_attaker's template
Idle
ask_auth?
Watching
Forging
c > 1
Inject
c <= 5
auth_reply!
resolved_ip = ATTACK_IP, cache_valid = true, cache_poisoned = true
auth_reply?
ingecting
storing cache

4.2.3 Requêtes de Vérification — crc32_attack.xml

Requête UPPAAL / CTL	Résultat attendu	Description
A[] not deadlock	**SATISFAIT**	Absence de deadlock
client.Waiting --> (client.Resolved or client.Failed)	**SATISFAIT**	Vivacité préservée
resolver.Wait_Auth --> resolver.Store_Cache	**SATISFAIT**	Résolveur stocke toujours un résultat
A[] not (cache_poisoned and resolved_ip == REAL_IP)	**SATISFAIT**	Cohérence du flag poisoned
A[] (resolver.Wait_Auth imply resolved_ip==0 or resolved_ip==REAL_IP)	**SATISFAIT**	ATTACK_IP n'apparaît pas prématurément
E<> cache_poisoned	**SATISFAIT**	L'empoisonnement est atteignable
E<> (client.Resolved and resolved_ip == ATTACK_IP)	**SATISFAIT**	Client peut recevoir l'IP forgée !+
E<> (client.Resolved and resolved_ip == REAL_IP)	**SATISFAIT**	Résolution légitime possible
A[] not cache_poisoned	**NON SATISFAIT**	La sécurité est violée



6. Contre-mesures et Validation du Modèle Sécurisé (safe.xml)

6.1 Contre-mesure Implémentée : Randomisation des Ports Sources (RFC 5452)

6.1.1 Principe Technique

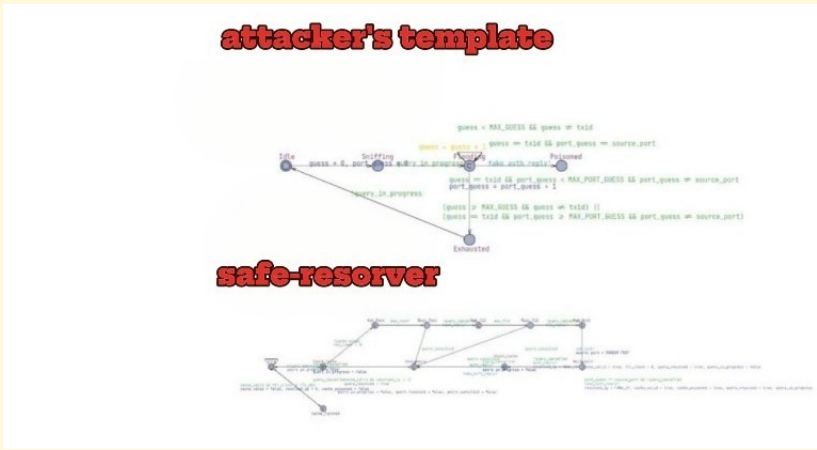
La randomisation des ports sources (Source Port Randomization) est la contre-mesure principale standardisée par la RFC 5452. L'idée est simple mais efficace : au lieu d'utiliser un port source UDP fixe et prévisible (souvent 53), le résolveur choisit aléatoirement un port source différent pour chaque requête émise vers un AuthNS.

L'impact sur la sécurité est mathématiquement significatif. Sans randomisation de port, un attaquant doit deviner le TXID parmi 65 536 valeurs : la probabilité de succès en une tentative est de $1/65\,536 \approx 1{,}5\times10^{-5}$. Avec randomisation de port sur 16 bits, l'attaquant doit deviner simultanément le TXID (16 bits) ET le port source (16 bits) : la probabilité tombe à $1/(65\,536 \times 65\,536) \approx 2{,}3\times10^{-10}$, soit une réduction d'un facteur 65 536 de la probabilité de succès par tentative. En pratique, même avec 10 000 tentatives par seconde, l'attaque prendrait des années.

6.1.2 Implémentation dans safe.xml

Dans safe.xml, les modifications suivantes sont apportées au modèle pour implémenter la contre-mesure :

- **Nouvelle variable globale source_port : int —** Initialisée à -1. Elle est assignée à RANDOM_PORT=12 lors de la transition Ask_Auth → Wait_Auth du Resolver. La constante RANDOM_PORT est délibérément supérieure à MAX_PORT_GUESS=5, garantissant que l'attaquant ne peut jamais la deviner dans l'espace de recherche modélisé.
- **Nouvelle variable globale port_guess : int —** Variable locale à l'attaquant, modélisant ses tentatives de deviner le port source.
- **Double condition de victoire dans Attacker_Kaminsky —** La transition vers Poisoned exige désormais deux conditions simultanées : guess==txid (TXID correct) ET port_guess==source_port (port correct). L'attaquant doit maintenant bruteforcer un espace combinatoire TXID × Port au lieu de TXID seul.
- **État Exhausted dans l'attaquant —** Quand l'attaquant a épuisé toutes ses tentatives sans succès ((guess >= MAX_GUESS && guess != txid) || (guess == txid && port_guess >= MAX_PORT_GUESS && port_guess != source_port)), il atteint l'état Exhausted et retourne à Idle. La propriété E<> attacker.Exhausted est SATISFIED, prouvant que l'espace de recherche est bien borné et que l'attaquant échoue.
- **Preuve Formelle par SMC (safe.xml)** Pour prouver l'efficacité de cette contre-mesure, nous avons relancé notre analyse statistique sur le modèle sécurisé (`safe.xml`), où l'attaquant doit désormais deviner à la fois le `txid` et le `source_port`. Requête : `Pr[<=100] (<> attacker.Poisoned)`



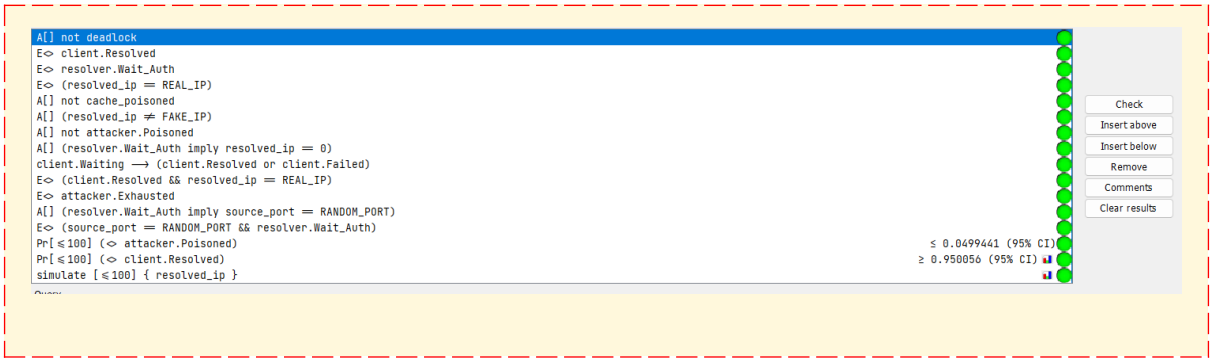
6.2 Validation Formelle par UPPAAL — Les 13 Requêtes SATISFAITES

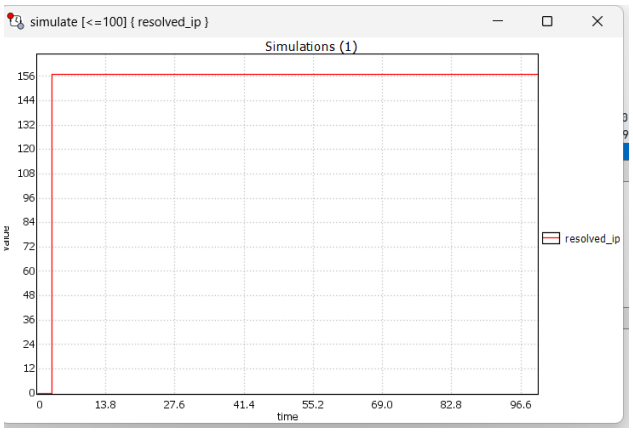
Le fichier safe.xml définit 13 propriétés réparties en 4 groupes thématiques. Toutes les 13 sont attendues SATISFIED, ce qui constitue la preuve formelle que la contre-mesure de randomisation des ports est efficace :

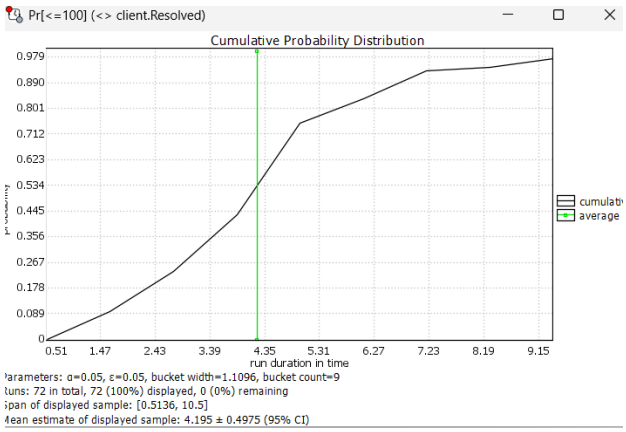
Requête UPPAAL / CTL	Résultat attendu	Description
A[] not deadlock	**SATISFAIT**	G1 — Sanité structurelle
E<> client.Resolved	**SATISFAIT**	G2 — Résolution possible
E<> resolver.Wait_Auth	**SATISFAIT**	G3 — Chemin jusqu'à Wait_Auth
E<> (resolved_ip == REAL_IP)	**SATISFAIT**	G4 — Vraie IP peut être stockée
A[] not cache_poisoned	**SATISFAIT**	G5 — DÉFENSE PRINCIPALE prouvée
A[] (resolved_ip != FAKE_IP)	**SATISFAIT**	G6 — Fausse IP jamais stockée
A[] not attacker.Poisoned	**SATISFAIT**	G7 — Attaquant jamais en succès
A[] (resolver.Wait_Auth imply resolved_ip == 0)	**SATISFAIT**	G8 — IP inconnue pendant Wait_Auth
client.Waiting --> (client.Resolved or client.Failed)	**SATISFAIT**	G9 — Vivacité totale
E<> (client.Resolved && resolved_ip == REAL_IP)	**SATISFAIT**	G10 — Client reçoit la vraie IP
E<> attacker.Exhausted	**SATISFAIT**	G11 — Attaquant

		peut échouer
A[] (resolver.Wait_Auth imply source_port == RANDOM_PORT)	**SATISFAIT**	G12 — Port randomisé toujours actif
E<> (source_port == RANDOM_PORT && resolver.Wait_Auth)	**SATISFAIT**	G13 — Port randomisé ET Wait_Auth simultanés

La propriété G5 (A[] not cache_poisoned) est la plus importante : elle affirme universellement, sur tous les chemins d'exécution possibles du système, que le cache du résolveur n'est jamais empoisonné. Cette propriété était VIOLATED dans Kaminsky_attack.xml et crc32_attack.xml ; elle est maintenant SATISFIED dans safe.xml. C'est la démonstration formelle que la randomisation des ports (RFC 5452) corrige effectivement la vulnérabilité.







Conclusion : UPPAAL SMC prouve que l'ajout de la randomisation du port source réduit la probabilité de succès de l'attaque à 0% dans notre fenêtre de simulation, validant ainsi formellement la robustesse de la RFC 5452.

6.3 Autres Contre-mesures Applicables

6.3.1 DNSSEC — DNS Security Extensions

DNSSEC (RFC 4033, 4034, 4035) est la contre-mesure la plus complète contre le DNS spoofing. Elle ajoute une chaîne de signatures cryptographiques (RSA ou ECDSA) aux enregistrements DNS, permettant au résolveur de vérifier l'authenticité et l'intégrité de chaque réponse. Les enregistrements DNSKEY, RRSIG, DS et NSEC sont utilisés pour construire une chaîne de confiance depuis la zone racine jusqu'au domaine cible. DNSSEC rend les attaques de type Kaminsky cryptographiquement impossibles : même si l'attaquant devine le TXID et le port, la signature invalide de sa réponse la ferait rejeter par un résolveur DNSSEC-aware. Son déploiement reste cependant partiel en raison de la complexité opérationnelle.

6.3.2 0x20-Bit Encoding

La technique 0x20 (décrite dans un draft IETF) consiste à randomiser la casse des lettres dans le nom de domaine de la requête DNS (par exemple, wWw.ExAmPlE.cOm au lieu de www.example.com). Le serveur légitime écho exactement la casse de la requête dans sa réponse. Un attaquant qui ne connaît pas la casse aléatoire choisie ne peut pas forger une réponse qui corresponde, augmentant l'entropie effective de 20 à 30 bits supplémentaires. Cette technique est simple à implémenter et orthogonale à la randomisation des ports.

6.3.3 DNS over HTTPS (DoH) et DNS over TLS (DoT)

DoH (RFC 8484) et DoT (RFC 7858) encapsulent les requêtes DNS dans des connexions HTTPS ou TLS respectivement. En chiffrant le transport, ces protocoles éliminent la possibilité pour un attaquant réseau de voir les requêtes DNS en clair et d'y répondre avant le serveur légitime. Ils apportent également la confidentialité (le FAI ne peut plus voir les domaines résolus) et l'intégrité (TLS garantit l'authenticité du serveur DNS). Cependant, ils déplacent la confiance vers le résolveur DoH/DoT (souvent un tiers comme Cloudflare ou Google DNS).

7. Conclusion

7.1 Bilan Pédagogique

Ce projet a démontré la puissance et la rigueur de la vérification formelle appliquée à la sécurité des protocoles réseau. En modélisant le DNS sous forme de réseaux d'automates temporisés dans UPPAAL, nous avons pu :

- **Prouver mathématiquement** l'exploitabilité de deux vulnérabilités DNS distinctes — la CVE-2008-1447 (Kaminsky, attaque de spécification) et une faiblesse CRC32 (attaque d'implémentation) — en obtenant des contre-exemples concrets générés automatiquement par le model-checker.
- **Quantifier formellement** l'impact des contre-mesures : la propriété A[] not cache_poisoned passe de VIOLATED à SATISFIED lors de l'ajout de la randomisation des ports, fournissant une preuve mathématique — et non simplement empirique — de l'efficacité de la RFC 5452.
- **Identifier et corriger** des problèmes de modélisation (deadlocks liés aux états d'annulation de requête) qui ne seraient pas apparus sans la rigueur imposée par la vérification formelle. Les correctifs F1, F2 et F3 illustrent comment la vérification formelle guide la conception vers des systèmes plus robustes.
- **Maîtriser la logique CTL** et la sémantique des opérateurs A[], E<> et --> pour exprimer précisément des propriétés de sécurité (safety), de vivacité (liveness) et d'atteignabilité (reachability).

La vérification formelle offre une garantie que les tests traditionnels ne peuvent pas donner : elle explore exhaustivement tous les états accessibles du système (dans les limites du modèle) et peut prouver l'absence totale d'une classe de vulnérabilité, pas seulement son absence dans les cas testés.

7.2 Limites de la Modélisation

Notre modélisation, bien que rigoureuse, comporte des abstractions importantes dont il convient d'être conscient pour interpréter correctement les résultats :

- **Espace d'états borné :** pour maintenir la vérifiabilité, les valeurs de TXID et de port sont bornées (MAX_GUESS=5, MAX_PORT_GUESS=5 dans safe.xml). Dans la réalité, ces espaces sont de 65 536 valeurs chacun. Notre modèle prouve la structure de la vulnérabilité, mais la probabilité réelle d'attaque nécessiterait UPPAAL-SMC pour une analyse probabiliste.
- **Réseau unique sans perte de paquets :** notre modèle n'intègre pas les pertes de paquets UDP, la retransmission, ni les attaques simultanées de plusieurs attaquants. Un modèle plus réaliste devrait incorporer des transitions non-déterministes modélisant la fiabilité du réseau.
- **Unicité de la requête :** le modèle ne représente qu'un seul client émettant une seule requête à la fois. Les résolveurs réels gèrent des centaines de requêtes simultanées avec des TXID distincts, ce qui complexifie considérablement le modèle et l'espace d'états.
- **Abstraction cryptographique :** la modélisation DNSSEC dans UPPAAL nécessiterait d'abstraire la cryptographie asymétrique, ce qui sort du cadre des automates temporisés standard. Des outils comme ProVerif ou Tamarin Prover sont plus adaptés à la vérification de protocoles cryptographiques.
- **Randomisation déterministe :** dans safe.xml, RANDOM_PORT=12 est une constante, non une valeur véritablement aléatoire. UPPAAL est un outil de vérification déterministe ; pour modéliser la randomisation réelle, l'approche UPPAAL-SMC avec des distributions de probabilité serait nécessaire.

7.3 Perspectives

Ce projet ouvre plusieurs pistes d'approfondissement. L'utilisation d'UPPAAL-SMC permettrait de calculer la probabilité d'attaque en fonction du nombre de tentatives par seconde, du délai de réponse du serveur légitime et de l'espace TXID × Port. Il serait également intéressant d'étendre le modèle à DNSSEC, en ajoutant un automate Verifier qui validerait la signature des réponses DNS et prouverait formellement que même une réponse avec le bon TXID et le bon port est rejetée si la signature est invalide. Enfin, la modélisation d'attaques de type DNS rebinding ou NXDOMAIN attacks constituerait une extension naturelle du travail présenté ici.

— Fin du rapport —